

Day 16 — decision trees and random forests

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 3, Day 2 — paired with [day16_trees_forests.py](#)

1. Beyond straight lines

Day 15's logistic regression finds one weighted combination of features that best separates survivors from non-survivors — essentially a straight-ish boundary through the data. It can't easily express "women survive at a high rate UNLESS they're in 3rd class AND traveling alone." Decision trees handle exactly that kind of interaction naturally. Same train/test split from Day 15, new model family.

2. How a tree decides

A decision tree asks a sequence of yes/no questions about the features ("Is Sex_male?", "Is Fare > 26?") and routes each passenger down a branch based on the answer, ending at a leaf that predicts survived or died. At each step, the tree picks the question that best separates survivors from non-survivors in the data it currently has — measured by Gini impurity, a score of how mixed a group still is.

3. Building a decision tree

```
from sklearn.tree import DecisionTreeClassifier

tree_unrestricted = DecisionTreeClassifier(random_state=42)
tree_unrestricted.fit(X_train, y_train)
```

Exact same interface as logistic regression — this consistency is deliberate in scikit-learn. Every model exposes `.fit()` and `.predict()` the same way, so swapping one model family for another rarely means rewriting the surrounding code.

4. Overfitting: a tree with no limits

An unrestricted tree keeps splitting until every leaf is perfectly pure — which usually means it has memorized the training set, including its noise, and performs worse on new data than a simpler tree would.

```
train_acc = accuracy_score(y_train, tree_unrestricted.predict(X_train))
test_acc = accuracy_score(y_test, tree_unrestricted.predict(X_test))
```

- In a real run: train accuracy 1.000, test accuracy 0.767 — a 0.233 gap. A perfect training score is itself a red flag, not something to celebrate
- `max_depth` caps how many questions deep the tree can go
- `min_samples_leaf` requires each leaf to cover a minimum number of passengers

- Limiting both brought train and test accuracy much closer together in testing (0.775 vs 0.817) — a smaller gap means the tree generalizes better, even though its training score looks less impressive

5. Random Forest: many imperfect trees, averaged

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=200, random_state=42)
forest.fit(X_train, y_train)
```

- Trains many trees, each on a random subset of rows and features
- Each tree overfits differently — averaging their votes cancels out most of that noise
- `n_estimators=200` means 200 separate trees get trained and their predictions combined by majority vote
- Important caveat found during testing: averaging reduces overfitting on average across many datasets, not guaranteed on every single run — on a small dataset, one random forest can still show a similar or even larger train/test gap than a single tuned tree

6. Feature importance

```
importances = pd.Series(
    forest.feature_importances_, index=X_train.columns
).sort_values(ascending=False)
```

Instead of a signed coefficient like logistic regression produces, tree models report how much each feature reduced impurity across all its splits, summed across every tree in the forest. It's not signed (no "pushes toward survival" direction), just a ranked measure of how useful each feature was for separating the two classes.

7. Comparing all three models

- Logistic regression: simple, interpretable, assumes roughly linear relationships
- Single decision tree: captures interactions, but unstable and prone to overfitting without depth limits
- Random forest: usually the strongest of the three on tabular data like this, at the cost of interpretability — though not guaranteed to win on every split, which is exactly why the script compares them side by side instead of assuming

On a small test set, close differences between models can flip with a different `random_state` — this is exactly why cross-validation exists, which is where Week 3 goes next.

8. Deliverable and push

```
git add day16_trees_forests.py
git commit -m "Day 16: Decision trees and random forests - overfitting, ensembles, fea
git push
```